

stm32 学习笔记

stm32 学习笔记

STM32 系统总览

STM32 器件选型

STM32 IO 分类

STM32 系统架构

STM32 寄存器映射

STM外设地址映射

外设寄存器

STM32 时钟系统

STM32 标准库系统

GPIO 的使用

GPIO 工作模式

GPIO 输出模式

GPIO 输入模式

GPIO 寄存器

输出数据寄存器 ODR

置位/复位寄存器 BSRR

输入数据寄存器 IDR

HAL 库中的 GPIO

STM32 时钟系统

1. HSE 外部高速时钟信号

2. PLL 时钟源

3. SYSCLK 系统时钟

4. AHB 总线时钟 HCLK

5. APB2 总线时钟 HCLK2

6. APB1 总线时钟 HCLK1

7.Cortex 系统时钟

8. RTC 实时时钟

STM32 中断

STM32 异常类型

NVIC 简介

中断优先级

NVIC中断编程

DMA 控制器

DMA 请求

ADC

ADC 概述

电压输入范围和几个 REF 引脚

ADC 通道选择

ADC 触发源

ADC 采样时间

ADC 数据寄存器

ADC 规则组数据寄存器 ADC_DR

ADC 注入组数据寄存器 ADC_JDRx

中断和事件触发

电压转换

ADC 工作模式

ADC 初始化结构体

DAC

数模转换和输出通道

DAC 初始化结构体详解

CAN 通信协议

CAN 通信物理层实现

CAN 协议层

CAN 协议中的中的差分信号

CAN 通信优先级和仲裁

CAN 的波特率

CAN 帧

CAN 数据帧的结构

帧起始 SOF

仲裁段

控制段

CRC

ACK

STM32 中的 CAN

一些常用的缩写

笔记中的寄存器均来自于 STM32G474

STM32 系统总览

STM32 器件选型



STM32/STM8社区
www.stm32.org

STM32 IO 分类

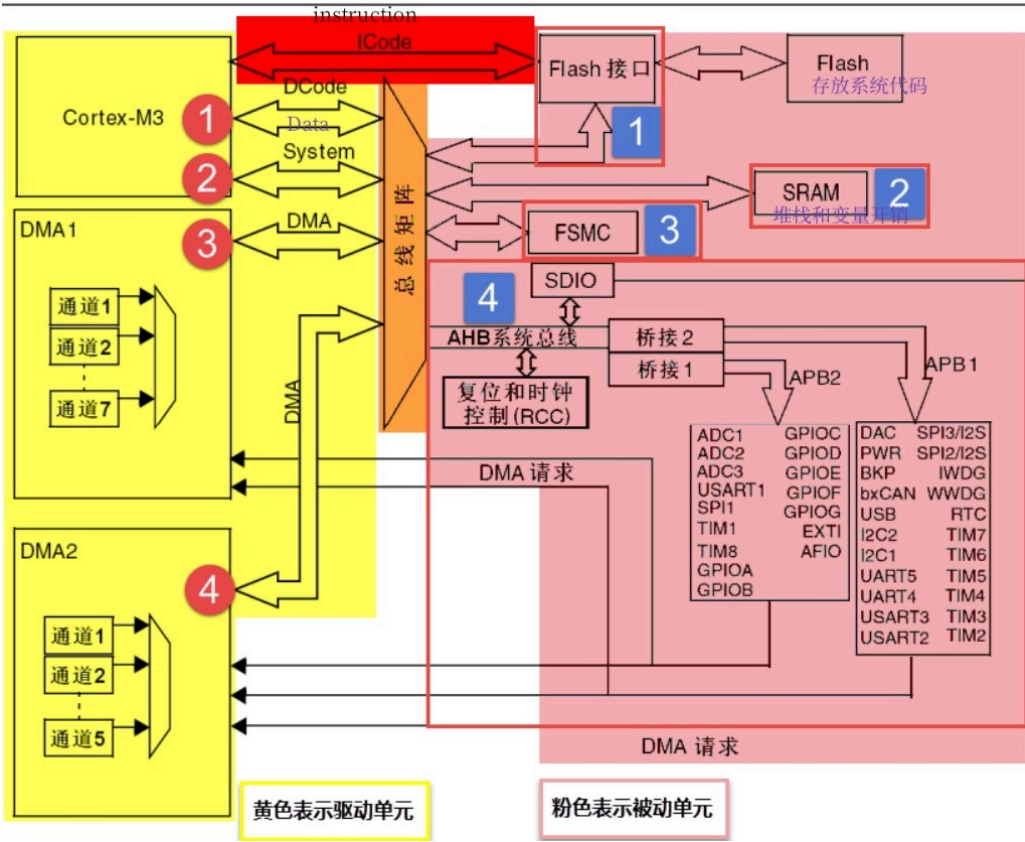
IO 分类	IO 用途
电源	VBAT、VDD、VSS、VDDA、VSSA、VREF
晶振	主晶振 IO、RTC 晶振 IO
下载	JTCK等
BOOT IO	BOOT 0、BOOT 1，用于设置系统的启动模式
复位	NRST，用于外部复位
GPIO	连接某些特定的外设器件

其中 BOOT 0 和 BOOT 1 设置系统的启动类型，即程序从哪里开始执行

BOOT 0	BOOT 1	程序起始位置
0	×	用户写入程序
1	0	系统存储器，BOOT Loader，又称为 ISP 程序

BOOT 0	BOOT 1	程序起始位置
1	1	内置 SRAM，用于快速调试程序

STM32 系统架构



STM32 含有 4G 个地址空间，其中以 512M 为块被分配给了不同的单元。因此可以引出 STM32 的外设地址映射。

STM32 寄存器映射

将已经给定的寄存器地址按照其具体功能给出其别名，称为寄存器映射。

STM 外设地址映射

片上外设挂载在三条总线上，APB1 上挂载低速外设，APB2 和 AHB 上面挂载高速外设，三条总线也有基地址，不同外设的地址偏移是相对于其上一层来讲的。特定外设的首个地址称为该外设的基地址。

外设寄存器

9.4.1 GPIO port mode register (GPIOx_MODER) (x =A to G)

Address offset:0x00
Reset value: 0xABFF FFFF (for port A)
Reset value: 0xFFFF FEBF (for port B)
Reset value: 0xFFFF FFFF (for ports C..G)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Note: It is recommended to set PB8 to a different mode than the analog one to limit the consumption that would occur if the pin is left unconnected.

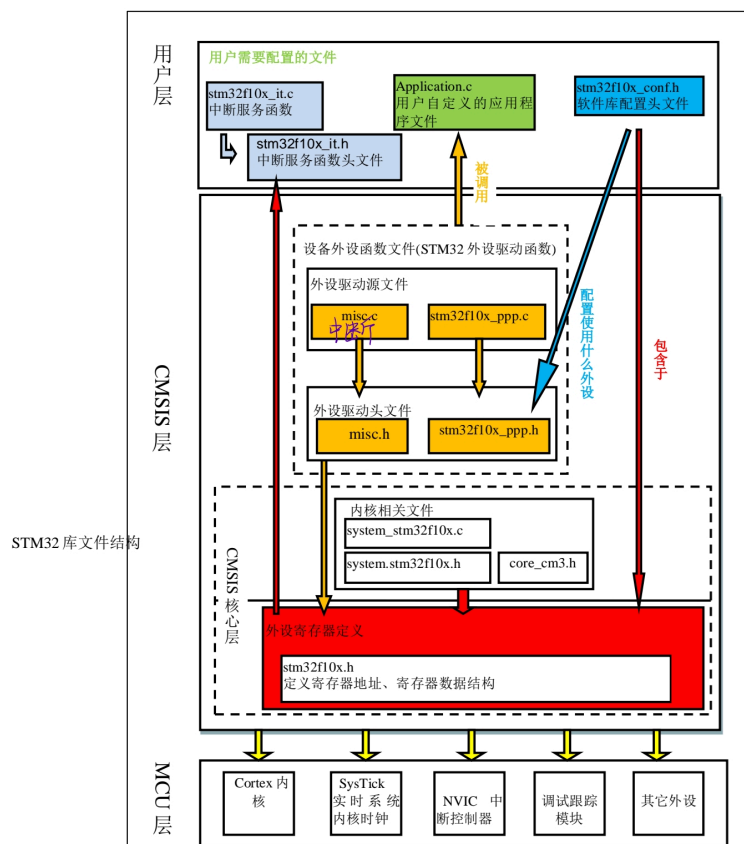
以 STM32G474 的GPIO 寄存器为例，最上方的是寄存器的名称，Address offset 是地址偏移量，可以根据外设基地址和地址偏移量获得该寄存器的地址，Reset Value 是复位值，指的是当电源复位时寄存器复位的值，rw 是指该位可读可写，Mode 用来描述寄存器的功能，介绍了寄存器中每一位的作用。

STM32 时钟系统

STM32 标准库系统

- CMSIS 标准库是一个 Cortex 内核和 ST 厂商之间建立的协议
- STM32 HAL 库下载地址：[STM32Cube MCU & MPU Packages - Products](#)，虽然说并不需要自己下载可以使用 Cubemx 配置，但是里面有一些手册可以查看，.chm 文件（库帮助文档）
- 启动文件 startup_stm32f10x.s：汇编文件，用于配置 STM32 的编译环境（？不懂，但是很重要）

- **stm32f10x_it.c**: 这个文件专门用来编写中断服务函数的，相关的中断服务函数接口可以在汇编文件中找到，具体查看 [STM32 中断](#) 这一部分
- **stm32f10x_conf.h**: 在实际使用的过程中，这个头文件用于包含使用的外设库的头文件，简化主程序中的代码，这个文件被包含在重要文件 **stm32f10x.h** 中，因此不需要重复包含，只需要编写即可
- **misc.c**: 用于配置外设与内核之间的 NVIC
- **system_stm32f10x.c**: 实现了 STM32 的时钟配置，启动文件中的 SystemInit 就是在这个文件中定义的
- 相关文件: STM32 用户手册、参考手册，其中具体的寄存器放在 STMxxxx 参考手册中，详细介绍了相关的用途等等，而用户手册可能更细致一些（？没有仔细读过），这两个文件直接搜就可以搜到，帮助文档 .chm 文件可以在上面的 HAL 库下载文件夹中间找到



GPIO 寄存器

输出数据寄存器 ODR

为上图中的 **Output control** 提供输入信号，可以通过修改此寄存器的值直接修改 GPIO 引脚的输出电平（当然前提是这个寄存器是可以被修改的，通过翻阅手册可以发现这个是可以直接修改的，其操作为 'rw'）

置位/复位寄存器 BSRR

可以通过修改 BSRR 中的值来操作输出数据寄存器中的值，可以使用 ^ 异或操作来对每一位求反

输入数据寄存器 IDR

可以通过 GPIO_IDR 读取 GPIO 的输入信号

HAL 库中的 GPIO

相关的 HAL 代码被存放在 `stm32f10x_gpio.h` 文件

通常往往只需要了解某个外设的初始化结构体就可以去了解到这个外设的具体功能。

STM32 时钟系统

1. HSE 外部高速时钟信号

可以通过有源和无源晶振产生，使用有源晶振时后时钟从 `OSC_IN` 进入，`OSC_OUT` 悬空，而使用无源晶振时 `OSC_IN` 和 `OSC_OUT` 都需要进入时钟信号，并且还要配置谐振电容。

2. PLL 时钟源

PLL 可以用来进行倍频，其时钟源来源可以来自 HSE，也可以来自 HSI/2，通过对 PLL 进行配置可以设置输出的时钟频率

3. SYSCLK 系统时钟

系统时钟可以来自于 HSE，HSI，PLLCLK

4. AHB 总线时钟 HCLK

SYSCLK 经过 AHB 预分频器后得到 AHB 总线时钟，也就是 HCLK

5. APB2 总线时钟 HCLK2

APB2 时钟 HCLK2 可以通过对 HCLK 进行 APB2 预分频得到，这条总线上面挂载的是高速设备

6. APB1 总线时钟 HCLK1

APB1 时钟类似于 HCLK2 时钟，但是频率较低，上面挂载的是低速设备

7. Cortex 系统时钟

Cortex 系统时钟通过对 HCLK 8 分频得到，可以用来驱动内核时钟 SysTick，该时钟信号一般用于系统的时钟节拍，也可用作普通的定时。

8. RTC 实时时钟

在掉电后仍旧可以运行，可为系统提供精确的计时

STM32 中断

中断：在主程序运行的过程中出现了特定的中断触发条件，使得 CPU 暂停当前正在运行的程序，转而去处理中断程序，处理完成后返回到原来的被暂停的位置继续运行。

STM32 异常类型

编号	优先级	优先级类型	名称	说明
	-3	固定	Reset	复位
	-2	固定	NMI	不可屏蔽中断，RCC 的时钟保护系统 CSS 会连接到这里
	-1	固定	HardFault	所有类型的错误
	0	可编程	MemManage	存储器管理
	1	可编程	BusFault	存储器访问失败
	2	可编程	UsageFault	未定义指令
	6	可编程	SysTick	系统滴答定时器

NVIC 简介

NVIC 是嵌套向量中断控制器，控制着整个芯片与中断相关的功能，是内核中的一个外设，NVIC 相关配置文件存放在 **misc.h** 文件中

中断优先级

NVIC 中有一个专门用来配置外部中断优先级的寄存器 NVIC_IPRx，是一个 8bit 的寄存器，但是 STM 厂商只使用了其中的高四位，称为表达优先级，同时表达优先级又可以被分为抢占优先级和子优先级

优先级分组	主优先级	次优先级	描述
NVIC_PriorityGroup_0	0	0-15	主0次4
NVIC_PriorityGroup_1	0-1	0-7	主1次3
NVIC_PriorityGroup_2	0-3	0-3	主2次2
NVIC_PriorityGroup_3	0-7	0-1	主3次1

NVIC中断编程

配置中断时候有三个步骤：

- a. 使能外部中断，具体由每个外设相关中断使能位控制
 - b. 初始化 NVIC_InitTypeDef 结构体，配置中断优先级分组
 - c. 编写中断服务函数
-

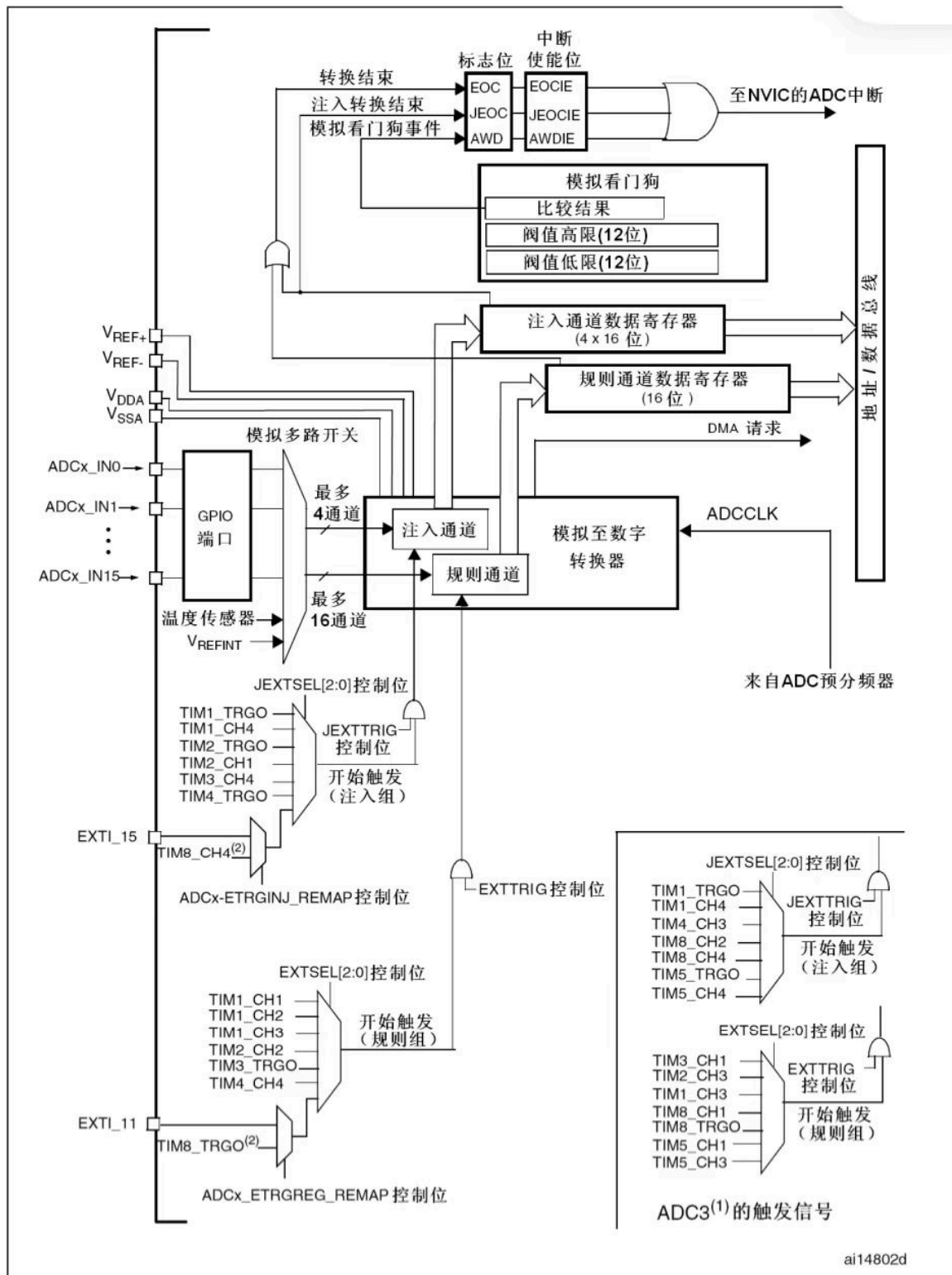
DMA 控制器

DMA（直接存储器存取）用来提供在外设和存储器之间或者在存储器和存储器之间的高速数据传输，而无须 CPU 的干涉。他也是一个外设，主要功能就是用来进行数据搬运。DMA 包含多个通道，通道可以理解为传输数据的一种管道。

DMA 请求

如果

ADC



ADC 概述

这里的 ADC 是 12bits 分辨率的。

- ADC 开关控制：通过设置 ADC_CR2 寄存器中的 ADON 可以给 ADC 上电。当这位为 0 时写入 1，可以将 ADC 唤醒，当这位为 1 时写入 1，可以启动转换，但是从转换器上电（ADC 第一次写入 1）到 ADC 开始转换（后面写入 1）中间有一个延迟时间，感觉其实并不需要关注
- ADC 时钟：RCC 控制器专门为 ADC 提供一个专用的可编程预分频器。

电压输入范围和几个 REF 引脚

ADC 的电压输入范围为： $V_{REF-} < V_{IN} < V_{REF+}$ ；其输入电压由 V_{REF-} 、 V_{REF+} 、 V_{DDA} 、 V_{SSA} 这几个外部引脚决定

引脚名称	信号类型	注释
V_{REF+}	输入，模拟参考正极	ADC 使用的高端/正极参考电压，输入值应该为 $2.4V < V_{REF+} < V_{DDA}$
V_{DDA}	输入，模拟电源[1]	等效于 V_{DD} 的模拟电源并且 $2.4V < V_{DDA} < V_{DD}$
V_{SSA}	输入，模拟电源地	等效于 V_{SS} 的模拟电源地
V_{REF-}	输入，模拟参考负极	ADC 使用的低端/负极参考电压，应满足 $V_{REF-} = V_{SSA}$ ，二者一般是接在一起的

[1]：根据 Deepseek 所言， V_{DDA} 是 ADC 模拟电路部分的电源供电电压，通常是和 V_{DDD} 分开进行设计的，将其独立是因为数字电源 V_{DDD} 的高频噪声可能会耦合到模拟信号当中，导致 ADC 的性能下降，而独立供电可以减少这种干扰，提高精度。

ADC 通道选择

每个 ADC 有多个通道，可以把转换分为两组：规则组和注入组：

规则通道：顾名思义，很规矩，按照顺序来进行转换

注入通道：注入就是插队的意思，可以在规则通道转换的时候强行插入转换，先转换注入通道，然后转换完成后再转换刚才的那个通道。

- 规则组由 16 个通道组成，规则通道和他们的转换顺序在 ADC_SQRx 寄存器中选择，规则组中的转换总数应该写入 ADC_SQR1 中的 SQL[3:0] 位中

- 注入组由 4 个通道组成，注入通道和转换顺序在 ADC_JSQR 寄存器中选择，总数应该写入到 ADC_JSQR[1:0] 位中

ADC 触发源

ADC 的开始转换可以通过对 ADC 控制寄存器 ADC_CR2 的 ADON 来控制，写入 1 的时候开始转换，写入 0 的时候停止转换。这里的 ADC_CR2

当然也可以通过使用触发转换，可以通过对 ADC_CR2 进行配置来开启。选择好触发源之后还可以选择触发源是否激活。

ADC 采样时间

ADC 可以通过对 ADC_SMPR 进行配置来设置采样时间，ADC 的转换时间为： $T_{conv} = \text{采样时间} + 12.5 \text{ 周期}$ 。

具体设置采样时间时候可以看 ADC_SMPR 寄存器中的相关描述。

ADC 数据寄存器

ADC 规则组数据寄存器 ADC_DR

ADC 规则组的数据寄存器只有 ADC_DR 一个，32bits，LSBs 是在单 ADC 时候使用，MSBs 是在 ADC1 中双模式下保存 ADC2 中的规则数据（双模式就是 ADC1 和 ADC2 同时使用）。因此在多通道使用的时候后面的数据可能会覆盖掉 ADC_DR 中的数据。要正常使用时要使用 DMA 进行数据搬移。

ADC 注入组数据寄存器 ADC_JDRx

注入组有 4 个通道，同时每个通道都有着自己的寄存器，不会像规则寄存器一样产生冲突。MSBs 保留，LSBs 使用。

中断和事件触发

转换结束中断	模拟看门狗中断	DMA 请求
规则、注入通道结束中断	当输入 ADC 的电压超过参考电压的范围时候触发	产生 DMA 请求将数据转移到内存当中

电压转换

通过 ADC 读出来的数据是一个 12bits 的数据，因此想要使用还需要经过一个转换。 $V = 3.3 * X / (2^{12})$

ADC 工作模式

单 ADC 有扫描模式和连续模式等等。双 ADC 时候还会有更多的细分模式。

- 扫描模式：若扫描模式为 **disable**，那么每次开启 ADC 转换都会只对规则通道中的第一个进行测量；若扫描模式为 **enable**，那么每次开启 ADC 转换会按照规则通道的顺序对 ADC 通道进行测量。
- 连续模式：连续模式就是指一轮测量完成后是否进入下一轮的测量。

多 ADC 工作时候，有多种模式可控选择：下面列了一些个人感觉重要的，具体可以参考博客：[STM32 双 ADC 详解](#)

- 同步规则模式：双 ADC 同时转换各自的规则组，二者同时进行的。
- 慢速交叉模式：二者交替转换一个规则组，一个完成后另一个等待 14 个时钟周期

还有一些乱七八糟的模式，可以参考上面的博客

ADC 初始化结构体

```
typedef struct
{
    uint32_t ADC_Mode;
    FunctionalState ADC_ScanConvMode;
    FunctionalState ADC_ContinuousConvMode;
    uint32_t ADC_ExternalTrigConv;
    uint32_t ADC_DataAlign;
    uint8_t ADC_NbrOfChannel;
} ADC_InitTypeDef;
```

- **ADC_Mode**: 配置 ADC 的模式，一个 ADC 时候是独立模式，两个 ADC 是双模式，双模式下面还有很多的模式可以选择，具体可以参见上一节 ADC 模式
- **ScanConvMode**: 是否开启扫描模式，扫描模式可以参考上面一节
- **ContinuousConvMode**: 是否开启连续模式
- **ADC_ExternalTrigConv**: 选择触发源，应该适合寄存器 CR2 中的相互关联的（翻了下手册和源代码，可能是这样的，😓）
- **DataAlign**: 数据对齐方式
- **NbrOfChannel**: 说明规则通道，注入通道的数目，实际是由两个参数的，这本书给合成一个表示了，存放到 ADC_SQR1 寄存器中的 SQL 中了

DAC

DAC 就是把输入的数字编码转换为对应的模拟电压输出，STM32F1 系列的 DAC 的分辨率可以配置为 **8 位**或者 **12 位**的数字输入信号。

图42 DAC 通道模块框图

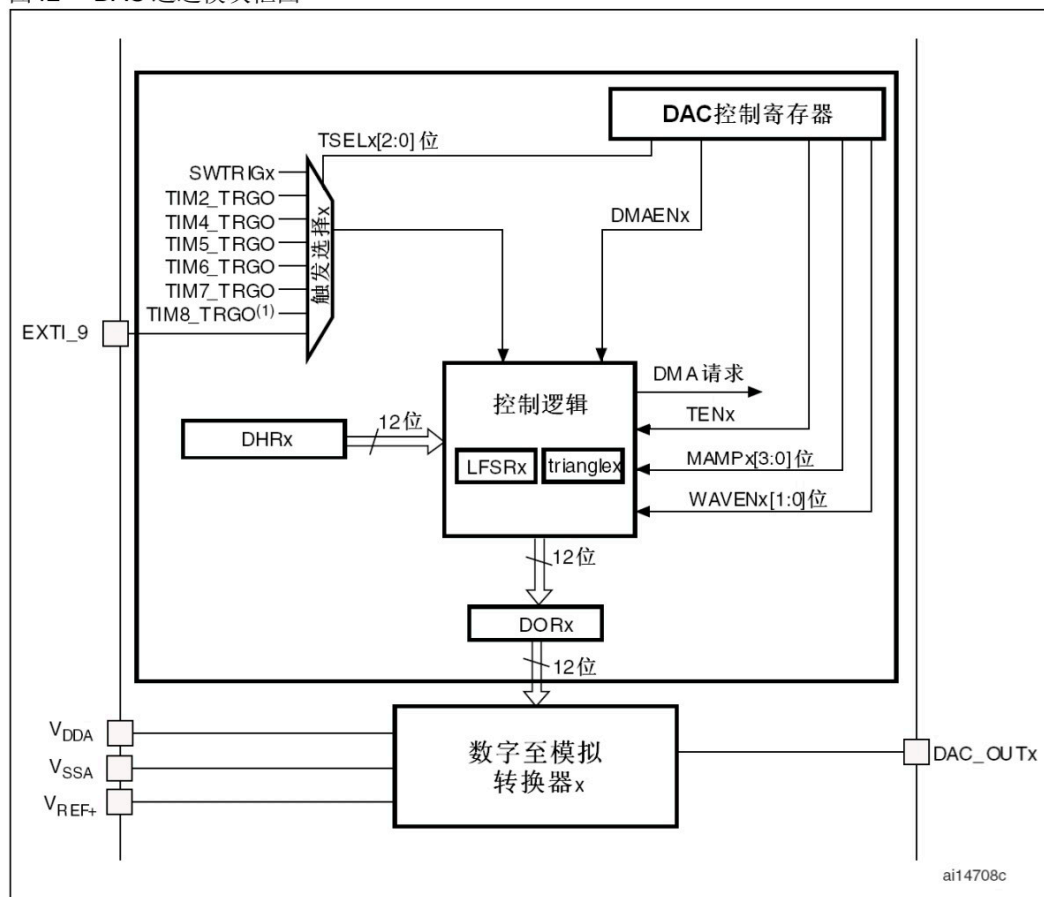


表70 DAC 引脚

名称	型号类型	注释
V _{REF+}	输入，正模拟参考电压	DAC使用的高端/正极参考电压， 2.4V ≤ V _{REF+} ≤ V _{DDA} (3.3V)
V _{DDA}	输入，模拟电源	模拟电源
V _{SSA}	输入，模拟电源地	模拟电源的地线
DAC_OUTx	模拟输出信号	DAC通道x的模拟输出

注意：一旦使能DACx通道，相应的GPIO引脚(PA4或者PA5)就会自动与DAC的模拟输出相连(DAC_OUTx)。为了避免寄生的干扰和额外的功耗，引脚PA4或者PA5在之前应当设置成模拟输入(AIN)。

整个框图都是围绕着下方的 数字至模拟转换器x 展开的，其输入为左边的参考电压和上面的数据寄存器 DORx，经过其转换后的数据通过右侧的 DAC_Outx 输出，上面的控制逻辑可以控制 DORx 加入伪噪声或配置产生三角波信号。左上角就是 DAC 的触发逻辑单元了。

数模转换和输出通道

DACx 是指有多个 DAC，每个 DAC 都有 1 个对应的输出通道连接到特定的引脚上面，**使用 DAC 功能时候引脚要配置成 模拟输入 AIN 模式，为了避免干扰。**转换时，数据不能直接写入到 DORx 中，要写入到寄存器 DHR12L1 或者 DHR12L2（12 代表位数，L 代表对其方式，1 代表 ADC1），当然也有双 DAC 的寄存器 DAC_DHR12RD 等等。

DAC 初始化结构体详解

```
typedef struct
{
    uint32_t DAC_Trigger;
    uint32_t DAC_WaveGeneration;
    uint32_t DAC_LFSRUnmask_TriangleAmplitude;
    uint32_t DAC_OutputBuffer;
}
```

- **DAC_Trigger:** 配置 DAC 的触发模式，当发生对应的触发事件的时候才会把 DHRx 寄存器中的值转移到 DORx 中去进行转换。
- **DAC_WaveGeneration:** 是否使用 DAC 输出伪噪声或者三角波，使能时候 DAC 会把 LFSR 中的数据叠加到 DHRx 上
- **DAC_LFSRUnmask_TriangleAmplitude:** 设置噪声生成器的低通滤波或者三角波的幅值
- **DAC_OutputBuffer:** 是否使能 DAC 的输出缓冲，使能了之后的 DAC 可以减小输出阻抗，适合驱动负载。

CAN 通信协议

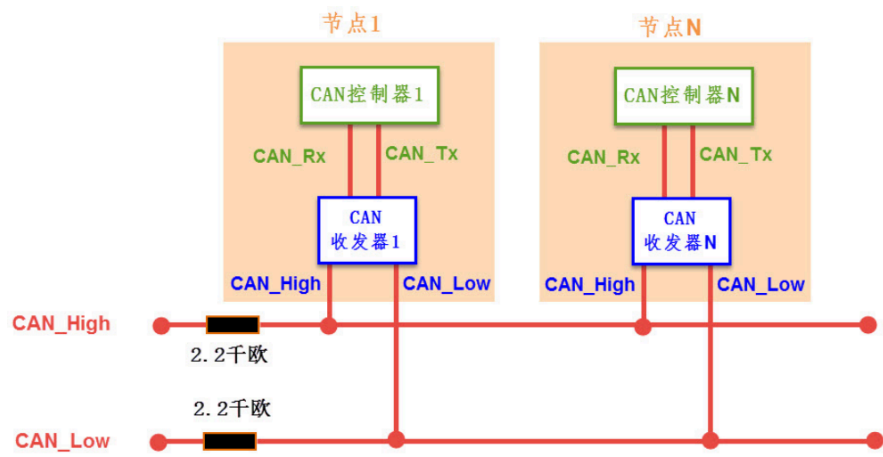
CAN 通信物理层实现

CAN 通信是一种异步通信模式，只具有 CAN_High 和 CAN_Low 两条信号线，以差分信号的形式进行数据的传输。所有设备挂载到总线上面进行通信。因此 CAN 是一种 半双工通信 协议。

CAN 通信有两种网络总线模式，一种是 闭环总线模式，需要在电路中加上两个 $120\ \Omega$ 的电阻，第二种是 开环总线模式，需要在电路中加上两个 $2.2k\Omega$ 大小的电阻，二者相比较结果如下：

性能	闭环总线模式	开环总线模式
电阻	120Ω	$2.2k\Omega$
通信速率	最高 1Mbps	最高 125kbps
通信长度	最高 40m	最高 1000m

CAN 通讯节点包含两部分，一个是 **CAN 收发器**，另一个是 **CAN 控制器**，CAN 收发器和 CAN 控制器之间通过 **CAN_Tx** 和 **CAN_Rx** 进行通信，CAN 收发器和 CAN 总线之间通过 **CAN_High** 和 **CAN_Low** 进行通信，而 CAN_High 和 CAN_Low 之间通过差分信号传输数据。CAN_Tx (Transmit) 数据线发送数据，CAN_Rx (Receive) 数据线接受数据。



CAN 协议层

CAN 协议中的中的差分信号

CAN 中的差分信号差值为 0 时表示逻辑 1，又称为隐性电平，差值为 2 V 时表示逻辑 0，又称为显性电平。多个设备同时发送时触发仲裁。

逻辑电平1	逻辑电平0
隐性电平	显性电平
CAN_High: 2.5 V	CAN_High: 3.5 V
CAN_Low: 2.5 V	CAN_Low: 1.5 V
0 V	2 V

CAN 通信优先级和仲裁

CAN 通信设备中每个节点都有对应的 ID，发送数据时先发送ID，当同时有多个设备发送时，显性电平可以盖过隐形电平，然后传输隐性电平的设备检测到线上出现争执，停止发送。

因此，CAN 通信中的标识符越低的发送节点具有越高的发送优先级，同时每个节点的 ID 应该为唯一确定的。

CAN 的波特率

CAN 通信属于 异步半双工 通信，因此需要配置好信号传输时的波特率，也就是说需要保证总线上的各个设备之间的 波特率相等 以确保通信的正常。

CAN 帧

CAN 帧被分为 5 种类型，分别是：数据帧、遥控帧、错误帧、过载帧、帧间隔

帧	帧用途
数据帧	用于节点向外传送数据
遥控帧	用于向远端节点请求数据
错误帧	向远端节点通知校正错误，请求重新发送上个数据
过载帧	通知远程节点：本节点尚未做好接收准备
帧间隔	将数据帧及遥控帧和前面的帧分离开来

CAN 数据帧的结构

数据帧分为标准数据帧和扩展数据帧，二者的区别主要集中在仲裁段，一个数据帧可以被分为多个部分。

帧起始 SOF

起始帧：CAN 空闲时总线上是隐性电平，SOF 将总线拉到显性电平，其他节点通过其跳变沿进行硬同步。

仲裁段

仲裁段主要为本数据帧的 ID，标准格式下 ID 为 11 位，而扩展格式下 ID 为 29 位。ID 低的优先级高，具体可以参考 [CAN 通信优先级和仲裁](#)

仲裁段组成	用途
ID	用于对信息进行仲裁
RTR	标准帧中用于区分数据帧和遥控帧
IDE	标识符拓展位，用于标识拓展帧和标准帧
SRR	代替标准帧中的 RTR，ID 相同时优先标准帧

因为 IDE 的存在，所以拓展数据帧的拓展 ID 要在 IDE 帧的后面。

控制段

控制段主要用来标识 DLC (DATA LENGTH CODE)，即数据长度码，用来标识数据段的字节数，因为数据段长度不定，为 0 ~ 8 bit

CRC

CRC 部分长为 15 bit，用于校验，当接受节点接收到的 CRC 码与自身计算不同时，将会发送错误帧。

ACK

ACK 段用于确保正确接收，发送节点填充隐性电平，接受节点若接受成功则发送显性电平以确保通信的正常进行。

STM32 中的 CAN

一些常用的缩写

缩写	中文含义
ISP	stm32 内置程序，boot loader